

Cloud Computing at AI Scale: Static vs. Reconfigurable Allocation in Google’s TPUv4 Supercomputer

Ankush Chandrashekar
CS218, Fall 2026

Code: <https://github.com/AnkushChandra/tpuv4-allocation-simulator>

September 30, 2026

Case study: Y. Zu et al., *Resiliency at Scale: Managing Google’s TPUv4 Machine Learning Supercomputer*, NSDI 2024 [1].

1 The Problem: Finding a Usable Set of Accelerators

Large training jobs are *gang scheduled*: every chip must be healthy at the same time, because a single failed process stops the synchronous weight update (Sections 1 and 2.1). In the static TPUv2/v3 pods, chips are wired in a fixed 2D torus, so a job must receive a *physically contiguous* block of chips that are all healthy and all free. If each machine is unavailable with probability p , a fixed block of N machines is usable with probability $(1 - p)^N$, which decays exponentially with job size. The paper calls this “combinatorially less likely as the system scales” (Section 2) and lists three causes: hardware outages and maintenance, fragmentation by other jobs (Figure 2), and deployment lead time (Section 2.1).

TPUv4 groups its 4096 chips into 64 *cubes* of 64 chips (16 machines, one rack). Forty-eight optical circuit switches (OCSes) can connect *any* set of healthy, free cubes into the torus a job asks for (Section 2.2). A failure removes one cube from the pool instead of breaking every block that contains it.

The cloud properties involved are **resource pooling** (whether idle capacity can actually be combined), **scheduling and fragmentation**, **fault tolerance and availability**, and **scalability**. Provisioning more accelerators does not help on its own: a job can only use capacity that is simultaneously healthy, unoccupied, and connected in the right shape.

2 The Simulator

The Python simulator (only `numpy` and `matplotlib`) models a 4096-chip pod as a 32×32 grid of machines with 4 chips each, grouped into 64 cubes of 4×4 machines.

- **Static policy:** the job needs a rectangle of whole-cube dimensions (the most square shape that fits, in either orientation) in which every machine is healthy and free. The rectangle may start at *any* machine offset, which is generous to the static design.
- **Reconfigurable policy:** the job needs any k cubes containing no failed or occupied machine.

- **Snapshot mode** (experiments 1–3) samples 5000 random pod states per data point and asks whether the job can start.
- **Event-driven mode** (experiment 4) runs a 30-day job 300 times while machines fail and are repaired. Under the static and reconfigurable policies an interruption loses the work done since the last checkpoint, and the job waits until it can be placed again. The hot-spare policy keeps that work and only pays a short migration delay, falling back to a checkpoint if no spare cubes are free.

Table 1 lists the parameters. As a check, the reconfigurable Monte Carlo result matches the exact binomial formula $P[\text{Bin}(64, (1 - p)^{16}) \geq k]$ in the unit tests and in Figure 1.

Parameter	Value	Source
Machine failure rate	0.08% per day	Fig. 12(a), §5.2
OCS failure rate	0.04% per day (48 OCSes, §2.2)	§5.2; Fig. 12(c)
Mean repair time	72 h (exponential)	assumption; not reported
Checkpoint interval / restart cost	2 h / 0.5 h	assumption
Hot-spare migration cost	0.1 h, no lost work	assumption, based on §7
Machine unavailability p (snapshot)	0.78% (fitted) or 0.24% (failures only)	Fig. 1 fit; $0.08\% \times 3$ days

Table 1: Simulator parameters.

3 Experiments

Experiment 1: Job size

Hypothesis: static success will fall steeply with job size, while reconfigurable success will stay near 100% until the job approaches the pod size. **Parameter changed:** job size, from 64 to 4096 chips.

Result (Figure 1b, $p = 0.78\%$): static success is 79% at 1024 chips, 27% at 1600, 6% at 2304 and 1% at 3072. Reconfigurable success stays at or above 99.9% up to 3072 chips (48 cubes), then falls to 67% at 3584 chips, 11% at 3840, and essentially 0 for the full pod.

Explanation: the static policy needs one specific shape to be completely clean. The reconfigurable policy only needs a *count* of clean cubes. On average $64(1 - p)^{16} \approx 56$ cubes are clean, so the reconfigurable cliff sits near 56 cubes. At the full pod both policies are identical, because every machine must be up.

Supported, with one refinement: reconfiguration moves the cliff rather than removing it.

Experiment 2: Required machine availability

Hypothesis: for large jobs, the reconfigurable pod will tolerate roughly ten times more machine unavailability, as the paper’s “99.9% vs. 99%” claim implies (Section 2.1). **Parameter changed:** machine availability, found by bisection as the lowest value that keeps job availability at or above 90%.

Result (Table 2, Figure 2): for a 3072-chip job, static needs 99.964% machine availability and reconfigurable needs 98.66%, so reconfigurable tolerates 37 times more downtime. For the full 4096-chip pod both need 99.99%.

Explanation: spare cubes absorb failures, but only when there are spare cubes.

Supported for jobs that leave slack. **Not supported** for a job that uses the whole pod.

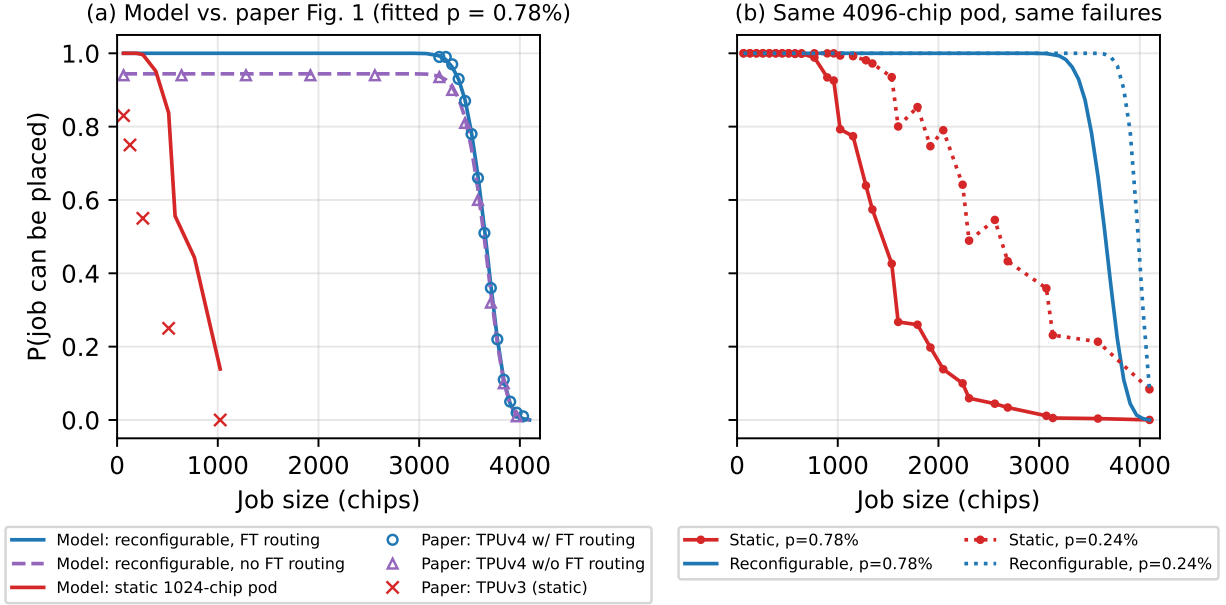


Figure 1: Experiment 1. (a) The model with one fitted parameter, against Figure 1 of the paper (points digitised by eye, about ± 2 percentage points). (b) Static vs. reconfigurable on the same 4096-chip pod at two machine unavailabilities.

Job size (chips)	Static needs	Reconfigurable needs	Downtime tolerated (reconf./static)
1024	99.43%	93.05%	12×
2304	99.961%	97.18%	73×
3072	99.964%	98.66%	37×
3584	99.981%	99.43%	29×
4096	99.990%	99.990%	1×

Table 2: Experiment 2: machine availability needed for 90% job availability.

Experiment 3: Other tenants and fragmentation

Hypothesis: scattered occupancy will block static placement long before the pod runs out of capacity, while reconfigurable placement will be unaffected until too few free cubes remain. **Parameter changed:** the share of cubes used by other jobs, placed either randomly or packed together. The job is 1024 chips and $p = 0.24\%$.

Result (Figure 3):

- With random placement, static success drops to 67% at 10% occupancy, 13% at 20%, and 0% from 40% onward. At 40% occupancy, 60% of the pod is still idle.
- Packing the other jobs together keeps static success at or above 87% up to 50% occupancy. At 60% it collapses to 0 even though 26 cubes are free, because they do not form a 4×4 block of cubes.
- Reconfigurable success stays above 99% until 70% occupancy.
- *Stranded capacity* here means free, healthy machines left over after placing the single largest job, not machines that no job of any size could use. It rises to 83% for static-random, because the largest rectangle leaves the rest behind. For reconfigurable it stays at 3.5%:

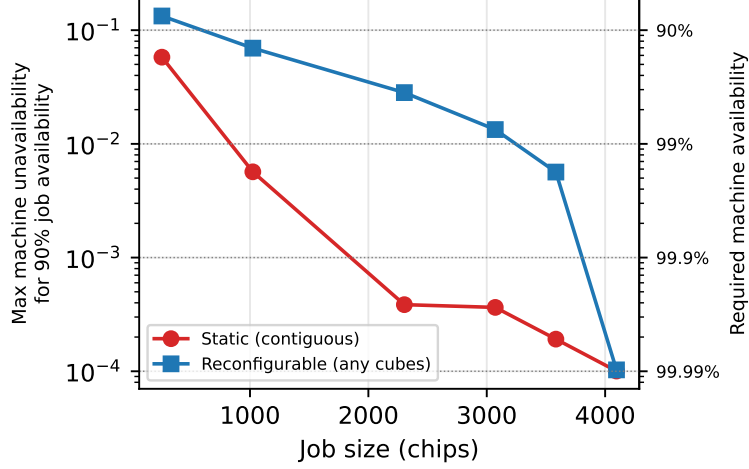


Figure 2: Experiment 2. Lower is stricter. The two designs meet at the full pod.

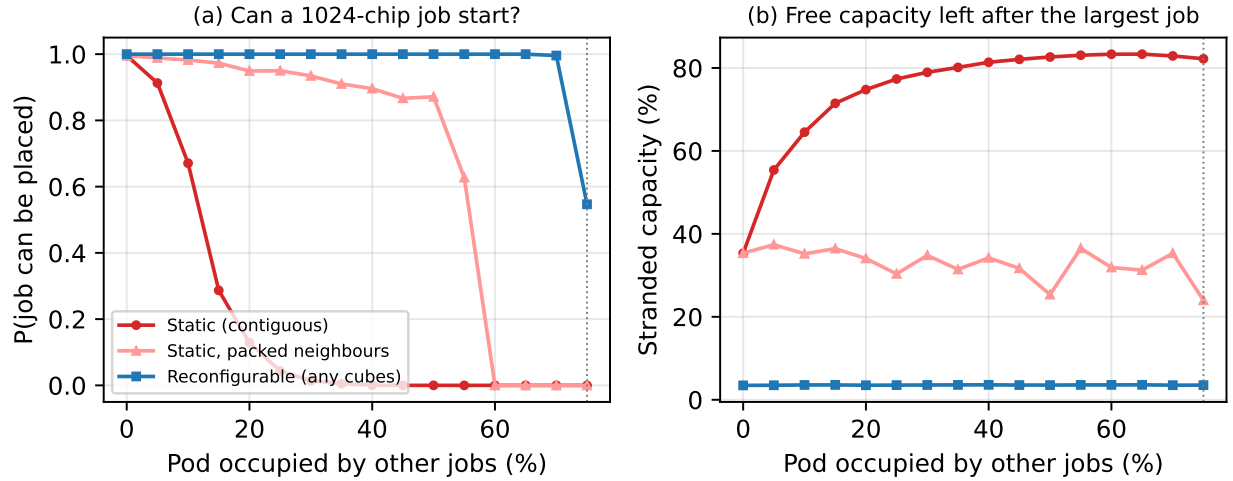


Figure 3: Experiment 3. The dotted line marks where free capacity equals the job size.

healthy machines trapped inside a cube with one failure.

Supported, with the caveat that packing requires a scheduler that preempts other jobs, as Borg does (Section 3.3).

Experiment 4 (not evaluated in the paper): Failures during a run

Hypothesis: interruptions will grow with job size. Reconfigurable jobs will recover in minutes while spare cubes exist, and hot-spare migration (proposed but not evaluated in Section 7) will add a few percent of goodput. **Parameters changed:** job size, policy and repair time. *Goodput* is useful time divided by wall-clock time.

Result (Table 3, Figure 4):

- A 3072-chip job is interrupted about 19 times in 30 days, roughly once every 38 hours.
- At that size the static job finishes in 84 days (39% goodput), the reconfigurable job reaches 96%, and the hot spare reaches 99.7%.

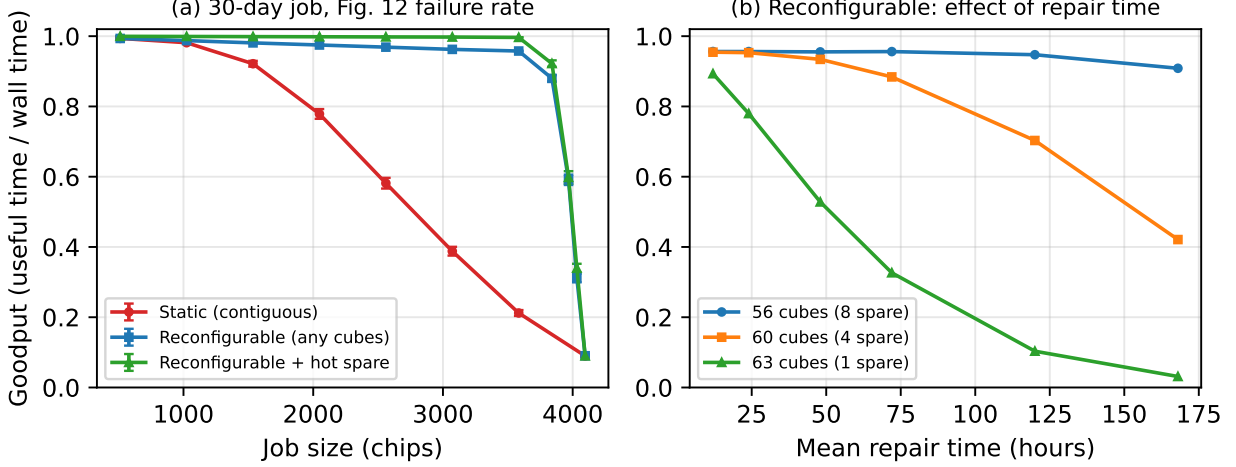


Figure 4: Experiment 4. (a) Goodput vs. job size. (b) Reconfigurable goodput vs. mean repair time.

- Goodput then collapses as spares run out: 88% with 4 spare cubes, 31% with 1, and 9% with none, when the “30-day” job takes a year.
- With 4 spare cubes, goodput falls from 95% to 42% as the mean repair time grows from 12 to 168 hours. With 8 spares it stays above 90%.

Explanation: about $1024 \times 0.08\% \times 3 \text{ days} \approx 2.5$ machines are down at any moment, usually in different cubes. A job that needs more than about 61 cubes therefore often finds no replacement and must wait for a repair. The hot spare removes lost work but needs the same spare cubes, so it cannot help near the cliff.

Partly supported: the collapse near the pod size was sharper than expected.

Job (chips)	Spare cubes	Static	Reconfig.	Hot spare	Interruptions / run
1024	48	98.1%	98.7%	99.9%	6
2048	32	77.9%	97.5%	99.8%	13
3072	16	38.8%	96.2%	99.7%	19
3584	8	21.2%	95.8%	99.6%	22
3840	4	–	88.0%	92.2%	24
4032	1	–	31.2%	34.0%	26
4096	0	9.0%	9.0%	9.0%	26

Table 3: Experiment 4: goodput of a 30-day job (300 runs per cell; 95% confidence intervals are at most ± 1.8 percentage points). A dash marks sizes that do not form a static rectangle.

4 Scalability Analysis

Adding resources does *not* always improve availability or performance.

- **Static pods:** each additional machine in a job multiplies its success probability by $(1 - p)$, so larger jobs become exponentially harder to place (Figure 1b). Hardware reliability requirements also tighten with size: 99.43% for 1024 chips and 99.96% for 3072 chips (Table 2).

- **Reconfigurable pods:** the bottleneck moves from “is there a clean contiguous block?” to “are there enough clean cubes?”. That creates a new cliff where the job size approaches the expected number of healthy cubes.

Experiment 4 shows the clearest example of scale adding cost. *Effective throughput* is job size times goodput:

- 53.6 cube-equivalents at 56 cubes;
- 52.8 at 60 cubes;
- 19.7 at 63 cubes;
- 5.8 at the full pod.

Beyond about 60 cubes, every extra cube reduces useful work. Keeping 4–8 cubes (6–12% of the pod) idle as spares is the price of resilience.

Other costs that scale brings:

- **Repair speed** becomes critical once spares are thin (Figure 4b).
- **Blast radius:** a single failure strands the other 15 machines in its cube (3.5% of healthy capacity in Experiment 3).
- **Paper-reported costs:** the OCS hardware is under 5% of capital cost (Section 2.2), and fault-tolerant routing slows jobs by 0.5–8.6% (Table 3).

5 Comparison with the Real System

What the simulator captures. Fitting the single parameter p to points read off the paper’s TPUv4 curve with fault-tolerant (FT) routing (Figure 1) gives $p = 0.78\%$, or 99.22% machine availability. The fit misses those digitized points by 0.35 percentage points on average (Figure 1a). That is not an error against a published table: the points themselves are only good to about ± 2 percentage points, as the caption notes. The *width* of the cliff, about six cubes from 90% to 10%, was not fitted: it follows from the binomial distribution of independent cube failures.

The paper’s 94% ceiling *without* FT routing is also reproduced from independent data. Using Figure 12(c)’s OCS failure rate and the same 3-day repair assumption, $(1 - 0.04\% \times 3)^{48} = 94.4\%$. The shared mechanism is that one OCS outage affects every cube (Section 5.2).

Where the results differ, and why.

- **Other sources of downtime.** The fitted p is about three times the unplanned-failure value of 0.24%. Planned maintenance, software upgrades and failed preflight checks (Section 3.6), none of which the model includes, probably account for the difference.
- **TPUv3 is badly under-predicted.** At the same p , the model says jobs of up to 192 chips on a 1024-chip static pod always fit, and a 512-chip job fits 84% of the time. Figure 1 does not tabulate those points, but its TPUv3 curve is already well below 100% for the smallest jobs shown and reaches 0 at 1024 chips. Failures alone cannot make such small jobs unavailable, so the TPUv3 data probably reflects load from other tenants (Experiment 3), older hardware, or pod-wide maintenance.

What the simulator leaves out:

- the 3D and twisted-torus topologies;
- link failures between cubes;
- correlated failures of a rack or power domain;

- job arrivals, priorities and preemption;
- routing performance.

6 Evaluating the Paper’s Claims

Claim 1: OCS reconfigurability keeps job availability high as jobs scale (Figure 1, Section 2).

- **Evidence:** production measurements from deployed pods. The TPUv4 curves run to 4096 chips. The TPUv3 curve stops at 1024 chips.
- **Rating: Moderate.**
- **Strengths:** the data is real, and the simulator confirms the mechanism on identical hardware (Figure 1b).
- **Weaknesses:** the comparison does not isolate the cause. TPUv3 and TPUv4 differ in hardware generation, pod size (1024 vs. 4096 chips), reliability and probably load. “Availability”, the measurement window and the sample size are undefined, and there are no error bars. The TPUv3 curve is already low for small jobs, which contiguity alone cannot explain, so some other factor is mixed in.
- **Experiment that would increase confidence:** schedule a TPUv4 pod in “contiguous-only” mode for a period and compare the two modes on the same hardware.

Claim 2: reconfiguration lowers the per-host availability needed for a 1024-host job from 99.9% to 99% (Section 2.1).

- **Evidence:** one sentence, with no derivation or data.
- **Rating: Weak.**
- **What the simulator shows:** it reproduces the *direction* and roughly the magnitude for large jobs that leave slack. At 3072 chips, static needs 99.96% and reconfigurable needs 98.66%.
- **Problem:** the paper says “1024 hosts” in a paragraph that also says “1024 chips,” so the unit is ambiguous. If a host is one machine, 1024 hosts is the whole TPUv4 pod, and the model shows *no* benefit: both designs need 99.99%. If a host is one chip, the model does show a benefit at 1024 chips, but the required availabilities are 99.43% and 93.05%, not 99.9% and 99%. Either reading needs spare capacity, which the paper does not state.

7 Public Cloud Perspective

Ideas that transfer well.

1. **Small failure domains plus $N+k$ spares.** Allocating in rack-sized units and holding a few idle nodes is a general availability principle; Experiment 4 suggests 6–12% spare capacity protects goodput. On AWS or GCP this means reserving slightly more capacity (capacity blocks or reservations) than the job uses. The abstraction helps because spares can be bought on demand, but they are paid for while idle.
2. **A detect–evict–reschedule loop.** Preflight health checks, automatic eviction, frequent checkpoints and fast restart (Sections 3.1 and 3.6) map onto Kubernetes node health checks, job controllers and checkpoints in S3 or GCS. None of this needs special hardware.

Ideas that are hard for a normal customer to reproduce.

1. **Topology reconfiguration and FT routing.** Rewiring the network optically and programming chip-to-chip routing tables requires owning the switches and the interconnect. The abstraction spares customers from cabling, but limits them: cluster placement groups are best-effort, a replacement node may land elsewhere in the network, and routing around a broken switch is impossible.
2. **Datacenter-level scheduling and repair priority.** Borg preempts jobs to defragment the pod (Section 3.3), and Google prioritises OCS repairs (Section 5.2). A customer can do neither, yet Figure 4b shows that repair time decides goodput when spares are thin. The provider hides repair work, but also the parameter the customer would most like to control.

8 Reflection

1. What building the simulator taught me. Reconfigurability does not remove the scaling cliff; it relocates it. Success is governed by the binomial tail around the expected number of clean cubes, $64(1 - p)^{16}$. That is why Figure 1’s TPUv4 curve is flat and then falls within about six cubes, and why the paper’s “99%” claim only works if the job leaves spare capacity.

2. What surprised me. How sharply goodput collapses near the pod size. Going from 60 to 63 cubes adds 5% more chips but cuts goodput from 88% to 31%, and a full-pod “30-day” job takes a year. About 2.5 machines are down at any moment, so with at most one spare cube the job spends most of its time waiting for repairs. The hot spare barely helps in that region, because it depends on the same spares.

3. The most important lesson.

- **General principle:** design for failure by making allocation independent of physical location, using small failure domains, and keeping $N+k$ headroom.
- **Infrastructure-specific technique:** OCS cube cross-connects combined with wild-first FT routing over the chip interconnect. It is effective, but only possible because Google controls the switches, the chips and the scheduler.

Disclosure of AI Use

AI was used to assist with the simulator and with understanding the paper.

References

- [1] Y. Zu, A. Ghaffarkhah, H.-V. Dang, B. Towles, S. Hand, et al. Resiliency at Scale: Managing Google’s TPUv4 Machine Learning Supercomputer. In *Proc. USENIX NSDI*, 2024.